

CS204 – Algorithms & Data Structures Laboratory

Dr. V. Vijaya Saradhi,
Dept. Of CSE,
IIT Guwahati

saradhi@iitg.ac.in



Tutorial #05 C++ Inheritance



Overview



- **Introduction**
- **Variety of Inheritances**
- **Inheritance Types And Access Rules**
- **Examples**
- **Inheritance & Constructors**
- **Base Class Member Functions & Their Behaviors**
- **Type Casting**
- **Functions & Base Class Objects**
- **Inheritance & Operator Overloading**

Introduction



- Sometimes, we want to express commonality between classes
- We want to create new class from existing class by adding new members or replacing (i.e., hiding/overriding) existing members
- Inheritance is a form of software reuse
- We create a class that absorbs an existing class's data and behaviors and **enhances them with new capabilities.**
- The new class should inherit the members of an existing class
- This existing class is called the **base class**
- The new class is referred to as the **derived class.**

Introduction



- A derived class represents a more specialized group of objects
- A derived class contains behaviors inherited from its base class and can contain additional behaviors.
- A derived class can also customize behaviors inherited from the base class.
- A **direct base class** the base class from which a derived class explicitly inherits
- An **indirect base class** inherited from two or more levels up in the class hierarchy.
- In the case of **single inheritance** a class is derived from one base class.

Introduction



- **In the case of multiple inheritance, a derived class inherits from multiple (possibly unrelated) base classes**
- **public, protected and private inheritance are possible**
- **Variety of inheritance.**



Variety of Inheritances





Variety of inheritances



Single Inheritance

```
class A
{
    private:
        int a1, a2, a3;
    protected:
        float p;
    public:
        A();
        ~A();
        f();
};

class B : public A
{
    private:
        int b1, b2, b3;
    public:
        B();
};
```




Variety of inheritances



```
class B
{
    private:
        int a1, a2, a3;
        Int b1, b2, b3;
    protected:
        float p;
    public:
        f();
        B();
};
```

Single Inheritance

A's members are all inherited
A's member functions are inherited

Variety of inheritances



Single Inheritance

```
class A
{
};

class B : protected A
{
};
```

Variety of inheritances

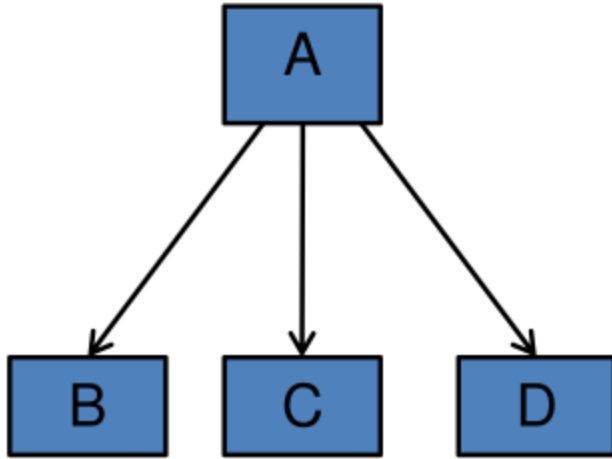


Single Inheritance

```
class A
{
};

class B : private A
{
};
```

Variety of inheritances



Hierarchical Inheritance

```
class A
{
};

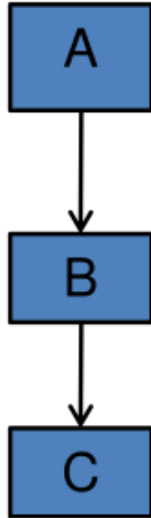
class B : public A
{
};

class C : public A
{
};

class D : public A
{
};
```



Variety of inheritances



Multilevel Inheritance

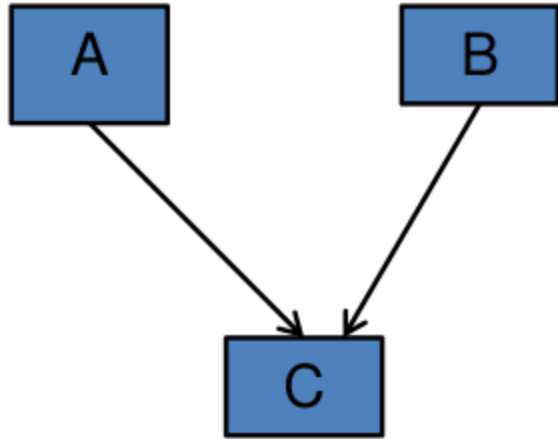
```
class A
{
};

class B : public A
{
};

class C : public B
{
};
```



Variety of inheritances



Multiple Inheritance

```
class A
{

};

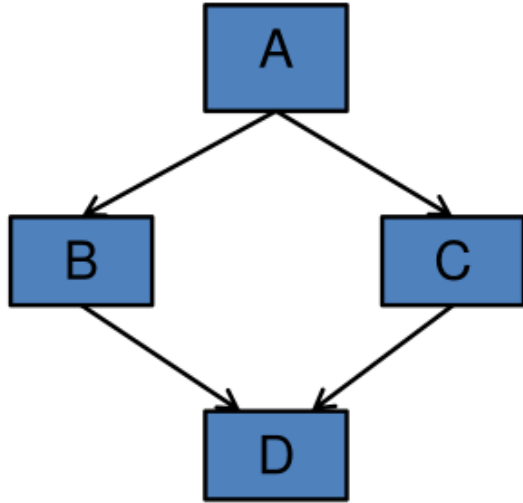
class B
{

};
class C : public B, public A
{

};
```



Variety of inheritances



Hybrid Inheritance

```
class A
{
};

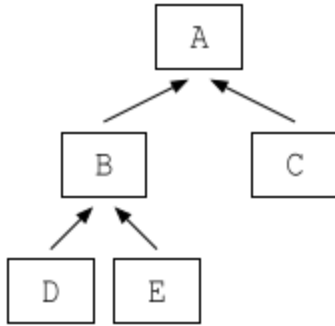
class B : public A
{
};

class C : public A
{
};

class D : public B, public C
{
};
```



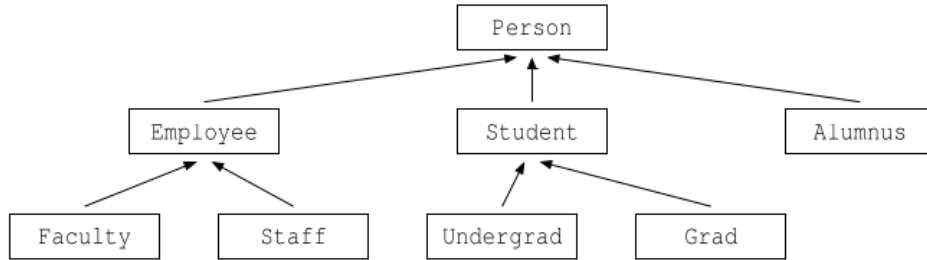
Variety of inheritances



```
class A
{
};
```

```
class B : public A
{
};
class C : public A
{
};
class D : public B
{
};
class E : public B
{
};
```

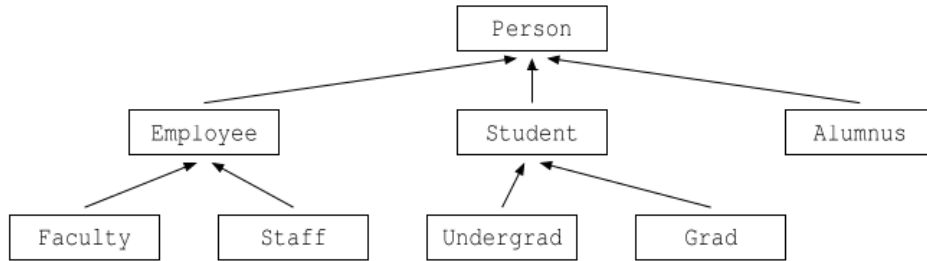

Variety of inheritances



```
class Person
{
};
```

```
class Employee : public Person
{
};
class Student : public Person
{
};
class Alumnus : public Person
{
};
```

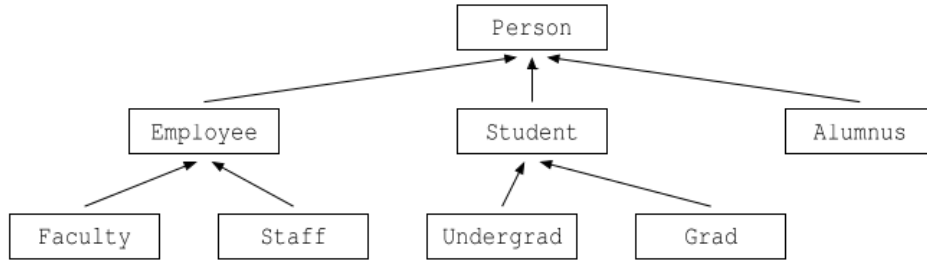
Variety of inheritances



```
class Employee : public Person
{
};
```

```
class Faculty: public Employee
{
};
class Staff : public Employee
{
};
```

Variety of inheritances



```
class Student : public Person
{
};
```

```
class Undergrad : public Student
{
};
class Grad : public Student
{
};
```



Inheritance Types And Access Rules



public inheritance



```
class Base
{
    public:
        void f();
    protected:
        void g();
    private:
        int x;
};

class Derived : public Base
{
    // f() is public
    // g() is protected
    // x is private and inaccessible from Derived
}
```

protected inheritance



```
class Base
{
    public:
        void f();
    protected:
        void g();
    private:
        int x;
};

class Derived : protected Base
{
    // f() is protected
    // g() is protected
    // x is private and inaccessible from Derived
}
```

private inheritance



```
class Base
{
    public:
        void f();
    protected:
        void g();
    private:
        int x;
};

class Derived : private Base
{
    // f() is private
    // g() is private
    // x is private and inaccessible from Derived
}
```



Examples



public inheritance



```
class Widget : public Derived
{
    public:
        void func_4()
        {
            func_2(); // OK
        }
};

int main(argc, char *argv[])
{
    Derived d;
    d.func_1(); // OK
    d.func_2(); // Error: protected
    d.x_0 = 10; // Error in accessible
}
```

```
class Base
{
    public:
        void func_1();
    protected:
        void func_2();
    private:
        int x_;
};

class Derived : public Base
{
    Public:
        Void func_3()
        {
            func_1(); // OK
            func_2(); // OK
            x_ = 0; // Error x_0: is private to Base
        }
}
```

protected inheritance



```
class Widget : public Derived
{
    public:
        void func_4()
        {
            func_2(); // OK
        }
};

int main(argc, char *argv[])
{
    Derived d;
    d.func_1(); // Error: protected
    d.func_2(); // Error: protected
    d.x_0 = 10; // Error: private
}
```

```
class Base
{
    public:
        void func_1();
    protected:
        void func_2();
    private:
        int x_;
};

class Derived : protected Base
{
    Public:
        Void func_3()
        {
            func_1(); // OK
            func_2(); // OK
            x_ = 0; // Error: x_0 is private to Base
        }
}
```

private inheritance



```
class Widget : public Derived
{
    public:
        void func_4()
        {
            func_2(); // Error: inaccessible
        }
};

int main(argc, char *argv[])
{
    Derived d; // OK. Public constructor
    d.func_1(); // Error: inaccessible
    d.func_2(); // Error: inaccessible
    d.x_0 = 10; // Error: inaccessible
}
```

```
class Base
{
    public:
        void func_1();
    protected:
        void func_2();
    private:
        int x_;
};

class Derived : private Base
{
    Public:
        Void func_3()
        {
            func_1(); // OK
            func_2(); // OK
            x_ = 0; // Error: x_0 is private to Base
        }
}
```



Inheritance & Constructors





Inheritance & constructors

- Constructors are not inherited

```
class A
{
    private:
        int a1, a2, a3;
    protected:
        float p;
    public:
        A();
        ~A();
        f();
};

class B : public A
{
    private:
        int b1, b2, b3;
    public:
        B();
};
```

```
class B
{
    private:
        int a1, a2, a3;
        Int b1, b2, b3;
    protected:
        float p;
    public:
        f();
        B();
};
```



Inheritance & constructors

- New members of derived class are not included in the base class
- That is b1, b2, b3 new data members in the derived class B are not part of class A.
- The data members a1, a2, a3 and p are inherited in class B
- Similarly, member function f() is inherited in class B

```
class A
{
    private:
        int a1, a2, a3;
    protected:
        float p;
    public:
        A();
        ~A();
        f();
};

class B : public A
{
    private:
        int b1, b2, b3;
    public:
        B();
};
```

Inheritance & constructors

- Since base-class constructors cannot initialize derived-class data members, inheriting constructors from base class by default would be bad idea
- This could lead to uninitialized data members
- For example, data members a1, a2, a3 and p are initialized in default constructor A().
- Class B has no constructor. Data members b1, b2 and b3 could go uninitialized and base class members are all initialized.
- A() cannot initialize b1, b2, b3



```
class A
{
    private:
        int a1, a2, a3;
    protected:
        float p;
    public:
        A() {a1=a2=a3=0; p=0.0}
        f();
};
class B : public A
{
    private:
        int b1, b2, b3;
    public:
};
```

Inheritance & constructors

- Default constructor, copy constructor and move constructor cannot be inherited
- Copy constructor A cannot be inherited



```
class A
{
    private:
        int a1, a2, a3;
    protected:
        float p;
    public:
        A(const A&)
        {
            a1=A.a1; a2=A.a2; a3=A.a3;
        }
        f();
};

class B : public A
{
    private:
        int b1, b2, b3;
    public:
};
```


Inheritance & constructors

- To inherit all other constructors, using statement must be invoked.

```
class B : public A
{
    public:
        int b1, b2, b3;
        // Following constructors NOT inherited
        // A(int k, int j)
        // A(int i, int j, int k)
};

int main(int argc, char *argv[])
{
    // Result in compilation error
    B box_01(10, 20, 30);
    B box_02(10, 20);
}
```

```
class A
{
    private:
        int a1, a2, a3;
    public:
        A(int k, int j)
        {
            a1 = k; a2 = j;
            a3 = k;
        }
        A(int i, int j, int k)
        {
            a1=i; a2=j; a3=k;
        }
};
```



Explanation



- Base class A has two constructors `A(int, int)` and `A(int, int, int)`
- Derived class B does not have constructor
- By default, class A constructors are NOT inherited
- Therefore, class B has no constructor
- In main the statement `B box_01(10, 20, 30);` tries to declare object `box_01` with a constructor which takes three arguments
- class B having no constructors this statement is illegal and result in compilation error

Explanation



- Base class A has two constructors `A(int, int)` and `A(int, int, int)`
- Derived class B does not have constructor
- By default, class A constructors are NOT inherited
- Therefore, class B has no constructor
- In main the statement `B box_01(10, 20);` tries to declare object `box_01` with a constructor which takes two arguments
- class B having no constructors this statement is illegal and result in compilation error

Inheritance & constructors

- To inherit all other constructors, using statement must be invoked.

```
class B : public A
{
    public:
        int b1, b2, b3;
        using A::A;
        // Following constructors are INHERITED
        // A(int k, int j)
        // A(int i, int j, int k)
};

int main(int argc, char *argv[])
{
    // OK
    B box_01(10, 20, 30);
    B box_02(10, 20);
}
```

```
class A
{
    private:
        int a1, a2, a3;
    public:
        A(int k, int j)
        {
            a1 = k; a2 = j;
            a3 = k;
        }
        A(int i, int j, int k)
        {
            a1=i; a2=j; a3=k;
        }
};
```



Explanation



- Base class A has two constructors A(int, int) and A(int, int, int)
- Derived class B does not have constructor
- "using A::A;" statement explicitly inherit A(int, int) and A(int, int, int) in class B
- Therefore, class B has two constructors from **base class**
- In main the statement B box_01(10, 20, 30); tries to declare object box_01 with a constructor which takes three arguments.
- class B has no constructor with three arguments; however inherited constructor A(int, int, int) is available.
- Therefore box_01 object is successful in creation. It calls base constructor A(int, int, int). Initializes a1, a2 and a3.
- The data members b1, b2 and b3 goes uninitialized



Constructors in both classes

- Writing identical constructors in both base class and derived class
- That is `A(int, int, int)` and `B(int, int, int)` as well as
- `A(int, int)` and `B(int, int)` is as follows
- This will initialize private members `a1`, `a2` and `a3` as well as `b1`, `b2` and `b3`.

Constructors in both classes



```
class B : public A
{
    private:
        int b1, b2, b3;
    public:
        B(int k, int j) : A(k, j)
        {
            b1 = k; b2 = j;
            b3 = k;
        }
        B(int i, int j, int k) : A(i, j, k)
        {
            b1 = i; b2 = j; b3 = k;
        }
};
```

```
class A
{
    private:
        int a1, a2, a3;
    public:
        A(int k, int j)
        {
            a1 = k; a2 = j;
            a3 = k;
        }
        A(int i, int j, int k)
        {
            a1=i; a2=j; a3=k;
        }
};
```



Constructors in both classes

```
int main(int argc, char *argv[])
{
    B box_01(10, 20, 30);
}
```

This leads to calling constructor of class B with three arguments B(int, int, int)

B(int, int, int) in turn calls A(int, int, int) explicitly and initializes private data members of class A

Followed by private data members of class B are initialized



Base Class Member Functions & Their Behavior



Base class member functions and their behavior



```
class Base
{
    // Assume members
public:
    void print()
    {
        cout << "a1: " << a1 << " ";
        cout << "a2: " << a2 << " ";
        cout << "a3: " << a3 << endl;
    }
};
```

```
class Derived : public Base
{
    // Assume members
public:
    // print() is not defined here
};

int main(int argc, char *argv[])
{
    Derived d;
    d.print();
}
```

As no `print()` member function is defined in the Derived class, Base class `print()` function is inherited

In the main function, a call to `print` through `d.print()` invokes base class `print` function

Base class member functions and their behavior



```
class Base
{
    // Assume members
public:
    void print()
    {
        cout << "a1: " << a1 << " ";
        cout << "a2: " << a2 << " ";
        cout << "a3: " << a3 << endl;
    }
};
```

```
class Derived : public Base
{
public:
    void print()
    {
        cout << "b1: " << b1 << " ";
        cout << "b2: " << b2 << " ";
        cout << "b3: " << b3 << endl;
    }
};

int main(int argc, char *argv[])
{
    Derived d;
    d.print();
}
```

Base class member functions and their behavior



- The print() function is implemented in both Base and Derived classes
- Depending on the invoked object, appropriate function is called
- Observed the following example

```
int main(int argc, char *argv[])
{
    Derived d;
    d.print(); //Derived print();

    Base b;
    b.print(); //Base print();
}
```

Base class member functions and their behavior



```
class Fruit
{
    public:
        void print()
        {
            cout << "Fruit class" << endl;
        }
};
```

```
class Apple : public Fruit
{
    public:
        void print()
        {
            cout << "Apple class" << endl;
        }
};
```

```
class Banana : public Fruit
{
    public:
        void print()
        {
            cout << "Banana class" << endl;
        }
};
```

```
int main(int argc, char *argv[])
{
    Fruit f; f.print(); //Fruit::print
    Apple a; a.print(); //Apple::print
    Banana b; b.print(); //Banana::print
}
```



Type Casting



Type Casting



- The statement `int j = 12; float f = (float) j;`
- Variable `j` is of type `int`
- It is type casted to `float`
- That is integer 12 to floating point 12
- Resulting value is assigned to variable `f`
- You will not observe any issues in general but for extreme values.



Base Class Object Assigned To Derived Class Object



Type Casting



- The following type casting is not allowed in classes
- `Example Base b; Derived d = b;`
- This is not allowed. It results in compilation error.
- The reason is that the Base class object `b` does not have the members `d1`, `d2`, `d3` of Derived class.
- Therefore, there's no meaningful way to copy `b` into `d` because the derived class-specific members cannot be initialized from `b`.
- The compiler sees that `b` is of type `Base`, and `d` is of type `Derived`. Since there's no implicit way to convert a `Base` object into a `Derived` object, this operation is not allowed.

Type Casting



```
#include<iostream>
using namespace std;
class Base
{
    private:
        int b1, b2, b3;
    public:
        Base()
        {
            b1 = 10; b2 = 20; b3 = 30;
        }
        void print()
        {
            cout << "b1: " << b1 << " ";
            cout << "b2: " << b2 << " ";
            cout << "b3: " << b3 << endl;
        }
};
```

```
class Derived: public Base
{
    private:
        int d1, d2, d3;
    public:
        Derived()
        {
            d1 = 100; d2 = 200; d3 = 300;
        }
        void print()
        {
            cout << "d1: " << d1 << " ";
            cout << "d2: " << d2 << " ";
            cout << "d3: " << d3 << endl;
        }
};
```

Type Casting



```
int main(int argc, char *argv[])
{
    Base b;
    b.print();
    Derived d;
    d = b;  
    d.print();
}
```

This results in compilation errors.

Type Casting



```
int main(int argc, char *argv[])
{
    Base b; b.print();
    Derived* dp = static_cast<Derived*>(&b);
    dp->print();
}
```

```
saradhi@Betaal:~/courses/cs204/july-dec-2024/tutorials/week06$ g++ inheritance-05.cc -g
saradhi@Betaal:~/courses/cs204/july-dec-2024/tutorials/week06$ ./a.out
b1: 10 b2: 20 b3: 30
d1: -741619200 d2: -926382803 d3: 2060400720
saradhi@Betaal:~/courses/cs204/july-dec-2024/tutorials/week06$
```

You can force the type casting but results in un-initialized values



Type Casting – Correct Method

```
class Derived: public Base
{
    private:
        int d1, d2, d3;
    public:
        Derived()
        {
            d1 = 100; d2 = 200; d3 = 300;
        }
        Derived(const Base &b)
        {
            d1 = 1000; d2 = 2000; d3 = 3000;
        }
        void print()
        {
            cout << "d1: " << d1 << " ";
            cout << "d2: " << d2 << " ";
            cout << "d3: " << d3 << endl;
        }
};
```

A type conversion constructor must be defined in the Derived class

The construct accepts a Base class reference.

The assignment
Derived d = b;

works as this conversion constructor is called during the assignment

```
int main(int argc, char *argv[])
{
    Base b; b.print();
    Derived d = b;
    d.print();
}
```



Derived Class Object Assigned To Base Class Object



Type Casting



```
int main(int argc, char *argv[])
{
    Derived d;
    d.print();
    Base b = d;
    b.print();
}
```

The above type casting will work.

Assigning derived class object to base class object will work

Type Casting



```
int main(int argc, char *argv[])
{
    Derived d;
    d.print();
    Base *b = &d;
    b->print();
}
~
```

The above type casting will also work.

Assigning derived class object to base class object will work



Functions And Base Class Objects





Functions & Type Casting

```
void function_1(Base &b)
{
    b.print();
}

int main(int argc, char *argv[])
{
    Base b;
    Derived d;

    function_1(b);
    function_1(d);
}
```

The above type casting will also work.

Assigning derived class object to base class object will work



Functions & Type Casting

```
void function_2(Base *b)
{
    b->print();
}

int main(int argc, char *argv[])
{
    Base *b = new Base();
    Derived *d = new Derived();

    function_2(b);
    function_2(d);
}
```

The above type casting will also work.

Assigning derived class object to base class object will work



Inheritance & Operator Overloading



Inheritance & Operator Overloading



- Overloaded operators are inherited by derived classes in C++, just like any other member function.
- If you define an overloaded operator as a member function in a base class, it will be inherited by any derived class.
- This means that the derived class can use that operator as if it were defined in the derived class itself.
- The inherited operator will behave according to the logic defined in the base class.
- However, the operator will work with the data members of the derived class if it is applied to an instance of the derived class.

Inheritance & Operator Overloading



- Just like any other member function, you can override an overloaded operator in a derived class by defining the operator function in the derived class.
- If the derived class overrides an operator but still wants to use the base class's version, it can explicitly call the base class's operator using the scope resolution operator (`Base::operator=`).

Inheritance & Operator Overloading



```
#include <iostream>
using namespace std;

class Base
{
public:
    int x;

    Base(int val) : x(val)
    {
    }

    Base operator+(const Base& other) const
    {
        return Base(x + other.x);
    }

    void display() const
    {
        cout << "Base x: " << x << endl;
    }
};
```

```
class Derived : public Base
{
public:
    int y;

    Derived(int v1, int v2) : Base(v1), y(v2)
    {
    }

    Derived operator+(const Derived& d) const
    {
        return Derived(x + d.x, y + d.y);
    }

    void display() const
    {
        cout << "Derived x: " << x << ", y: " << y << endl;
    }
};
```

Inheritance & Operator Overloading



```
int main()
{
    Base b1(5), b2(10);
    Base b3 = b1 + b2;
    b3.display();

    Derived d1(3, 7);
    Derived d2(4, 6);
    Derived d3 = d1 + d2;
    d3.display();

    Base b4 = d1 + d2;
    b4.display();

    return 0;
}
```

Base b3 = b1 + b2;

Calls Base class operator+ function

Derived d3 = d1 + d2;

Calls Derived class operator+ function

Base b4 = d1 + d2

Calls Derived class operator and assigns only x value in b4



Questions?





Thank You

